DPUCANBUS

DPUCANBUS represents a driver for CAN bus cards. Several DPUs can access the CAN bus at the same time. DPUCANBUS manages all access to one CAN bus channel. DPUs sending CAN messages assign control over the correct timing to DPUCANBUS, which sends the messages periodically. Additionally, DPUCANBUS receives all CAN messages and forwards them to the DPU that requested the message.

1. Static parameters:

CANMode

Selects the hardware (CAN bus card) which is used to communicate over the bus.[1]

Mode	CAN hardware
0	Software simulation of the CAN bus (local host only)
1 - 2	*PEAK PCI card channel 1 – 2 (#)*
3	*PEAK USB card channel 1 (#)*
4 - 5	ETAS/VECTOR PCMCIA-CardXL channel 1 - 2
6 - 7	VECTOR Board XL channel 1 – 2
8 - 9	VECTOR Card XL channel 1 – 2
10	VECTOR Card Y
11 - 12	VECTOR Card X channel 1 – 2
13	VECTOR Card 2
14	VECTOR CANpari
15 - 16	VECTOR CANAc2 channel 1-2
17 - 18	VECTOR CANAc2 PCI channel 1 – 2
19	VECTOR Case XL channel 1
21 - 22	*PEAK 2010 driver PCI channel 1 - 2 (#)*
23 - 24	*PEAK 2010 driver USB channel 1 - 2 (#)*
25 - 32	PEAK PCANBasic driver ISA BUS 1 - 8
33 - 40	PEAK PCANBasic driver PCI BUS 1 - 8
41 - 48	PEAK PCANBasic driver USB 1 - 8
49 - 50	PEAK PCANBasic driver PPC Bus 1 - 2
51 - 59	VECTOR XL driver channel 1 - 9
60	INTEL driver

[1] Different interpretation on Linux. Please see the „Notes for Linux“ below.

61-63	UDP/IP based simulation of the CAN bus (network-capable); channel 1 – 3
70-79	UDP/IP based communication PCAN-Gateway (network-capable); connection 1 – 10

Note: Modes marked with (#) are obsolete. The old PEAK drivers are superseded by the new PCANBasic driver (Mode=25..50). The old drivers are no longer supported in 64 bit versions of SILAB.

CANType Selects if the CAN message is interpreted as a standard message (=0; the default) or an extended message (=1)

Baud Communication speed of the CAN bus[2]

Baud	Actual speed
0	1000 kBaud
1	500 kBaud
2	250 kBaud
3	125 kBaud
4	100 kBaud
5	50 kBaud
6	33 kBaud
7	20 kBaud
8	10 kBaud
9	5 kBaud
10	don't care[3]

TraceChannel If set to *true*, creates a Vector ASC containing the processed CAN data. Default: *false*

TraceFileName File name for the Vector ASC file. Default: "test.asc"

Trace Enables recording to the Vector ASC file. If this flag is temporarily set to *false*, the recording is interrupted. Default: *true*

Config (only required for the UDP/IP based simulation of the CAN bus): a string in format *"<IP>:<PortSend>:<PortReceive>"*, which specifies the IP address of the communications partner and the UDP ports used for the communication.

2. Dependencies to other DPUs:

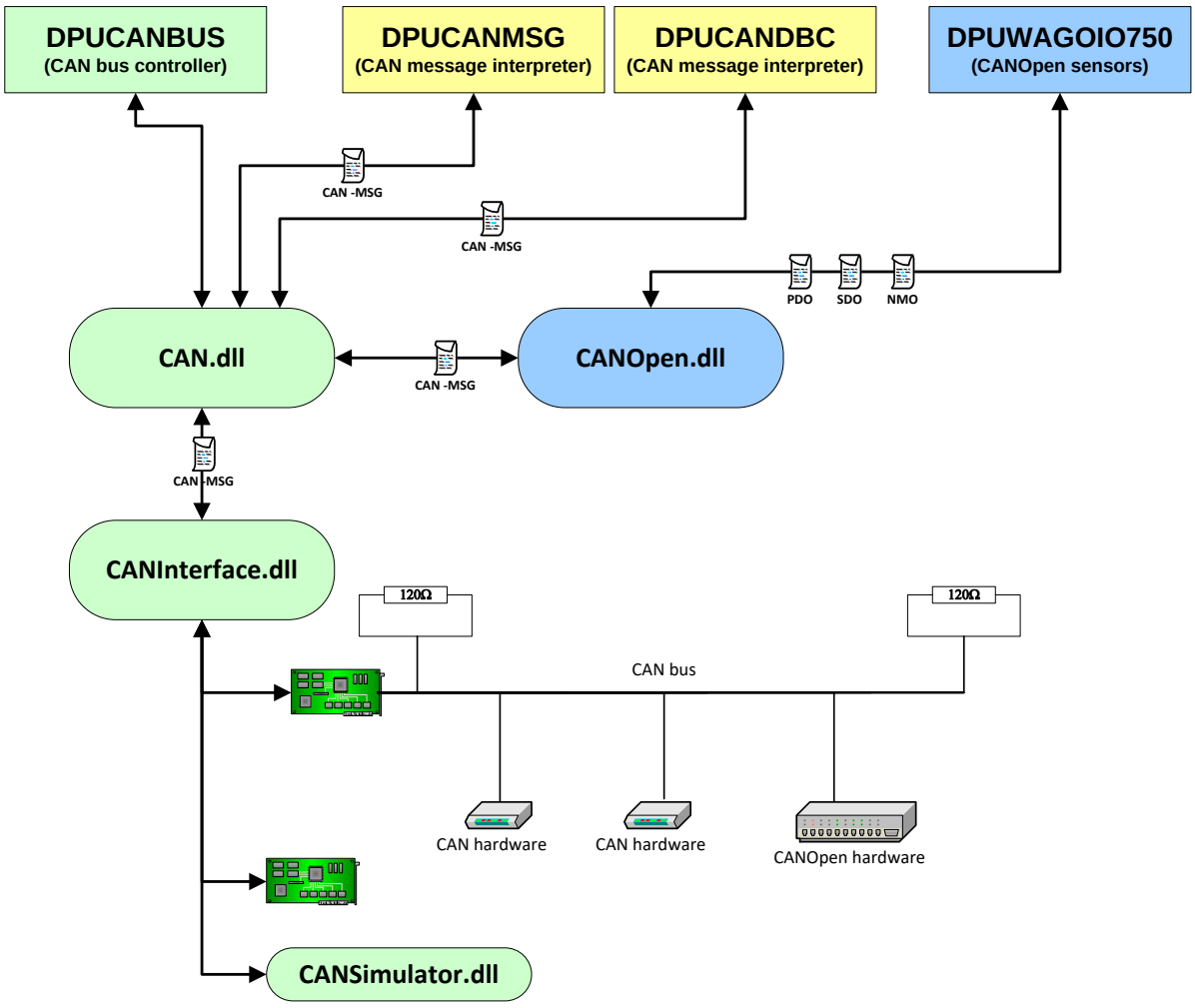
DPUCANBUS requires the libraries CAN.dll and CANInterface.dll. Every DPU that wishes to communicate over the CAN bus, requires an instance of DPUCANBUS with the same CANMode. The Index value of DPUCANBUS must be lower than the Index value of the DPU that communicates over the CAN bus.

[2] Not available on Linux. Please see the „Notes for Linux" below.

[3] This setting is only supported by some of the cards.

3. Functional principle:

DPUCANBUS manages one channel of a CAN bus card. The library CAN.dll collects all CAN messages (labeled CAN-MSG in the following graph) and submits them to the CAN bus according to their timing parameters. Incoming CAN messages are collected in the library CAN.dll and are forwarded to the corresponding recipients.



DPUs that use the CANOpen protocol, send and receive their PODs, SDOs and NMOs using the library CANOpen.dll. It transforms the CANOpen messages to corresponding CAN messages and then sends them over the CAN bus.

Instead of a CAN bus hardware, a software simulation of the CAN bus can be used. The library CAN-Simulator.dll simulates the CAN bus in the following way: Every CAN message that has been sent also received afterwards.

4. Notes for Linux-based operating systems:

On Linux-based operating systems, DPUCANBUS operates using the SocketCAN interface that is provided by the Linux kernel. This results in the following differences from the Windows version of the DPU:

- DPUCANBUS cannot activate the CAN bus by itself. (This operation requires superuser permissions; but SILAB shouldn't be run as the superuser.) Instead, the CAN bus(ses) must be activated in a system startup script using the following command:

  ```
  ip link set <IFID> up type can bitrate <BPS>
  ```

  For example, to bring up bus can0 (the first available CAN bus) with a speed of 500 Kbps:

  ```
  ip link set can0 up type can bitrate 500000
  ```

- The **Baud** parameter is ignored, because the baud rate cannot be changed by the DPU.
- The **CANMode** parameter is interpreted according to the following table:

Mode	CAN hardware
0	Software simulation of the CAN bus (local host only)
1 – 10	*Uses SocketCAN interface* **can0** … **can9**
11-13	*UDP/IP based simulation of the CAN bus (net-work-capable); channel 1 – 3*
14-23	UDP/IP based communication PCAN-Gateway (net-work-capable); connection 1 – 10

5. Example:

This sets up a local CAN bus (software simulation):

```
DPUCANBUS ~localCAN {
      Computer = { LOCALHOST };
      Index = 11;

      CANMode = 0;
      CANType = 0; # standard
      Baud = 1; # 500 kBaud
};
```